An AI Based High-Precision Industrial Thermometer

Puni Aditya, Vivek K. Kumar, Jnanesh Sathisha Karmar, Burra Shreyas Goud, Dr. G.V.V. Sharma
Department of Electrical Engineering,
Indian Institute of Technology Hyderabad,
Kandi, India 502284
gadepall@ee.iith.ac.in

***Abstract*—In this paper, we design and develop an industrial thermometer using the PT100 temperature sensor. This is done by analyzing the suitability of five machine learning (ML) techniques: a quadratic/inverse quadratic Least Squares (LSQ) model, a Stochastic/inverse Stochastic Gradient Descent (SGD) trained model, and a non-linear Random Forest Regressor (RFR) by calibrating them with respect to a commercial pen thermometer. The calibration process is also automated using Optical Character Recognition (OCR). Through numerical results, we conclude that the LSQ model is superior to the other models and implement it using the Arduino framework on an Arduino UNO.**

I. Introduction

Temperature measurement is a fundamental requirement in process control, automotive systems, and instrumentation. Among the various contact temperature sensors available, the Platinum Resistance Temperature Detector (RTD), commonly known as the PT100, is considered the standard for precision thermometry [1]. The PT100, are widely favored in industrial applications for their linearity and stability. [2] investigated the transient (dynamic) behavior of resistance temperature sensors using two experimental techniques: the Plunge Test and the Self-Heating Test. The goal was to identify the sensor's Transfer Function for use in real-time control systems. Here, the focus was only on the dynamic modeling (time delay/response) of the sensor, not the static (steady-state) calibration or non-linearity correction, which is critical for overall accuracy across a wide range. Futher, the above approach required specialized dynamic testing equipment. [3] used an Artificial Neural Network (ANN) to non-linearly calibrate PT100 sensor readings across −50°C to 150°C. In [4], Multisim software was used for the simulation and design of the measurement transducer. Both the above works were primarily based on software simulation only. They was did not implement or compare against machine learning models, relying solely on theoretical circuit performance, leaving the real-time processing feasibility and computational overhead on embedded hardware unaddressed. A high-accuracy temperature measurement circuit using a Wheatstone bridge, an AD623 differential amplifier, and an STM32F4 microcontroller was designed and implemented in [5]. However, the design relies on expensive, specialized analog components and a high-performance microcontroller.

To address the above issues, we compare the accuracy of three different regression approaches: the LSQ quadratic, grounded in the theoretical Callendar-Van Dusen equation [6], the SGD [7] and RFR [8] [9]. The coefficients, mathematical formulations, and validation accuracy for each model are presented, demonstrating that machine learning approaches can augment traditional calibration methods. In the process, we detail the calibration and characterization of a PT100 sensor by comparing two hardware signal conditioning setups: a simple voltage divider and a precision Wheatstone bridge interfaced with an ADS1115 ADC. We also automate the process of generating the dataset by using OCR to capture the readings of the analog thermometer through a mobile phone camera. The parameters for all the above ML techniques are finally hardcoded in the ATMEGA328P microcontroller, using the Arduino board. Through numerical performance plots, we show that LSQ outperforms other ML techniques considered in this paper.

II. PT-100 Fundamentals

The PT100 is defined by a nominal resistance of $100\,\Omega$ at 0°C. Unlike thermocouples which generate a voltage, RTDs differ in that they operate by changing electrical resistance in direct proportion to temperature changes [10]. This relationship is inherently non-linear over wide ranges, though it is highly predictable. The resistance $R(t)$ at temperature t is governed by the Callendar-Van Dusen equation [6]

$$R(t) = R_0(1 + At + Bt^2) \tag{1}$$

for temperatures $t > 0$°C, where A and B are standard coefficients. This quadratic physical relationship serves as the theoretical justification for using LSQ models in this work.

While PT100 sensors offer high stability, the resistance change is relatively small ($\approx 0.385\,\Omega/$°C). Consequently, accurate reading requires precise signal conditioning to convert resistance changes into measurable voltage levels [11]. This paper aims to optimize this conversion by analyzing different hardware front-ends and software regression models [12] to find an optimal solution for high-precision, low-cost temperature sensing.

III. Hardware Setup

Two main analog front-ends were tested to evaluate the trade-off between circuit complexity and measurement accuracy. Table I lists the components required for both analog front-ends, detailing the specific parts used for high-accuracy and cost-effective implementations.

Component	Qty.	Description
Arduino Uno	1	Microcontroller for processing and control.
ADS1115 ADC	1	16-bit external ADC for high-precision voltage measurement.
PT100 RTD Sensor	1	Platinum resistance thermometer.
100Ω Resistor	1	Precision resistor for Wheatstone bridge.
4kΩ Resistors	2	Resistors for Wheatstone bridge.
JHD 162A Parallel LCD	1	For displaying the final temperature.
22kΩ Potentiometer	1	To adjust the contrast of the LCD.
Breadboard	1	For creating solderless circuit connections.
Jumper Wires	Set	For connecting all the components.
Pen Thermometer <= 0.1°C resolution	1	Used for training purpose and as a reference.
Kettle/Heater with water	1	To heat the water, so that training data corresponding to a wide range of temperatures can be obtained

TABLE I: Component List

- *High-Precision Wheatstone Bridge with ADC:* This uses a Wheatstone bridge configuration coupled with an external 16-bit ADS1115 ADC [13]. This setup allows for differential measurement, significantly reducing noise and common-mode errors. The connection details for integrating the external 16-bit ADS1115 ADC are available in Table II

ADS1115 Pin	Arduino Pin	Purpose
VDD	5V	Power Supply
GND	GND	Common Ground
SCL	A5 (SCL)	I2C Clock Line
SDA	A4 (SDA)	I2C Data Line
A0	Input from Voltage Divider Node (between PT100 and R_REF)	Reading Voltage
A1	Junction between the two 4kΩ resistors	Reference for differential reading

TABLE II: ADS1115 Connections

The complete circuit schematic for the high-precision Wheatstone bridge setup is illustrated in Fig. 1. The complete circuit schematic for the simple voltage divider setup is illustrated in Figure 2.

- *Simple Voltage Divider:* This uses a simple voltage divider utilizing the Arduino's internal 10-bit ADC.

The Arduino-LCD connections are available in Table III.

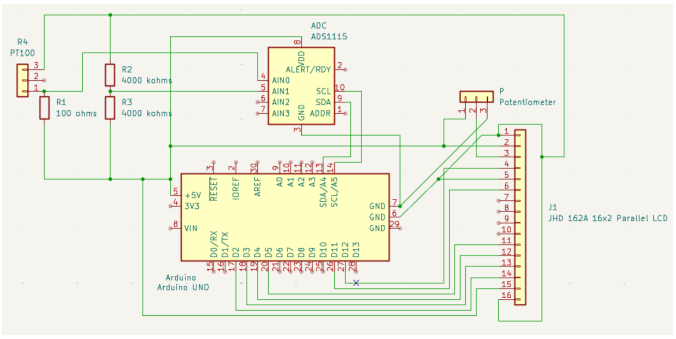

Fig. 1: Circuit Schematic of the High-Precision Wheatstone Bridge (Setup 1)

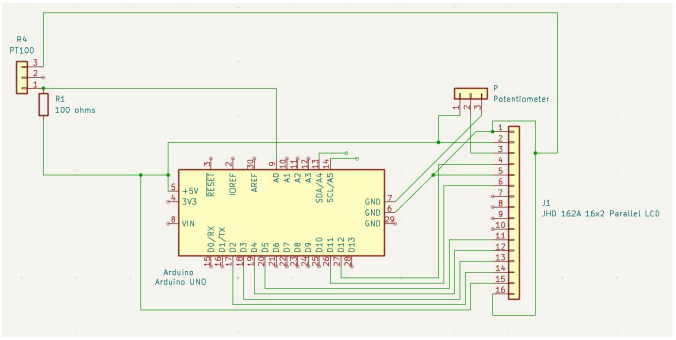

Fig. 2: Circuit Schematic of the simple voltage divider (Setup 2)

TABLE III: LCD Connections

LCD Pin	Arduino Pin
VSS	GND
VDD	5V
V0	Potentiometer Middle Pin
RS	Digital 12
RW	GND
E	Digital 11
D4	Digital 5
D5	Digital 4
D6	Digital 3
D7	Digital 2
A (Anode)	5V
K (Cathode)	GND

The code for the Arduino, which reads the voltage across the PT-100 and displays it into the LCD and flashing instructions is available here https://github.com/gadepall/pt100/codes/arduino/datacollection

IV. Automated Training Using OCR

The dataset is collected by simulating a real-world thermal change (cooling cycle) to capture paired temperature readings. Since the training data is available only on an analog thermometer, instead of manually noting the temperature, we automate the data collection process using OCR based techniques. Appendix A has more information on the software execution.

A. Experimental Setup

The experimental setup is available in Fig. 3. The practical requirements are listed below

1) *Hardware:*
   - The arduino setup with the code from the previous section flashed.
   - A phone connected to your computer.
   - (Optional) An NVIDIA GPU with CUDA installed. The script is pre-set to use `gpu=True` for much faster processing.
2) *Software:*
   - Python 3.x
   - The required Python libraries: `opencv-python`, `easyocr`, and `numpy`.

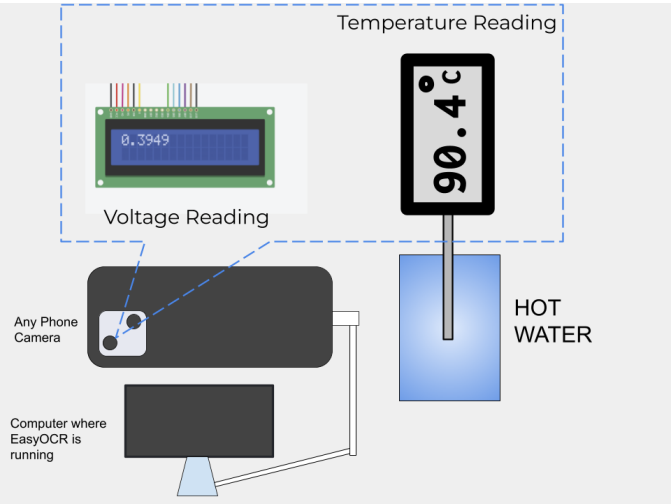

Fig. 3: Auto-Trainer representation

B. Data Collection Methodology

1) Setup: Place both the PT100 probe and the reference thermometer inside an electric kettle, submerged in water.
2) Heating Phase: Turn on the kettle and heat the water to its boiling point.
3) Recording Phase:
   - Switch off the kettle at boiling point.
   - Start recording a video using the smartphone.
   - Ensure both the PT100 LCD display and reference thermometer are visible in the frame.
4) Cooling Phase: Continue recording the cooling process as the water temperature drops naturally.
5) Data Extraction: The recorded video acts as the raw dataset.
6) Extract frames and apply OCR to detect readings from both displays.

V. Machine Lerning Models

A. Model Explanations

The LSQ and SGD models obtained from the dataset derived in the previous section are available in Table IV. The coefficients are determined through linear least squares regression [12].

Model	Equation	a	b	c	Description
Inverse LSQ (No ADC)	$X = aY^2 + bY + c$	−0.000014990	0.005589496	2.526066163	Inverse Least Squares with voltage divider setup without ADC.
Inverse LSQ (ADC)	$X = aY^2 + bY + c$	−0.000006744	0.004344843	0.056019368	Inverse Least Squares with ADC setup and Wheatstone bridge.
Quadratic LSQ (ADC)	$Y = aX^2 + bX + c$	147.376208110	197.568472526	−10.357480491	Least Squares (Quadratic) for Wheatstone bridge with ADC.
SGD	$Y = aX^2 + bX + c$	190.226107678	173.380527679	−7.072446221	Stochastic Gradient Descent model fit.
Inverse SGD	$X = aY^2 + bY + c$	−0.000005386	0.004183549	0.060452454	Inverse Stochastic Gradient Descent model fit.

TABLE IV: X denotes Voltage and Y denotes Temperature.

1) For inverse LSQ and SGD methods, to predict Y from a given X, we use the quadratic formula

$$Y = \frac{-b \pm \sqrt{b^2 - 4a(c - X)}}{2a} \tag{2}$$

   This method aligns with the physical properties described by the Callendar-Van Dusen equation [6]. The SGD Regressor is a linear model trained with Stochastic Gradient Descent [7] [9]. Due to its sensitivity to feature scaling, both the input features (X) and the target variable (Y) were scaled using StandardScaler before training. The predictions were then inverse-transformed back to the original scale for evaluation.
2) *Standard LSQ and SGD:* This model applies a standard quadratic fit directly mapping voltage to temperature, expressed as

$$Y = aX^2 + bX + c. \tag{3}$$

   Unlike the inverse models, this approach predicts temperature (Y) directly from the voltage (X) without requiring the use of the quadratic formula for inversion during the prediction phase.
3) *Random Forest Regressor:* The Random Forest Regressor is an ensemble learning method that constructs a multitude of decision trees during training [9]. For regression tasks, it averages the predictions of individual trees to produce a more stable and accurate prediction. This model was applied to predict temperature (Y) from voltage (X) directly, leveraging its ability to capture complex non-linear relationships without explicit feature engineering.

The models are obtained and implemented using Python and the scikit-learn [9] library. The implementation of these models and the evaluation of Mean Squared Error

(MSE) and R-squared (R2) metrics are available here https://github.com/gadepall/pt100/blob/main/codes/models/PT100_models.ipynb

VI. Model Evaluation and Comparison

Table V summarizes the Mean Squared Error (MSE) and R-squared (R2) scores for all evaluated models [14]. Based on these metrics, the LSQ (Quadratic) model for the Wheatstone bridge with ADC outperforms the rest. Therefore, the voltage divider method without external ADC is not suitable for our design.

Model	MSE	R2
1. Inverse LSQ (Volt. Div w/o ADC)	0.6612	0.9977
2. Inverse LSQ (Wheatstone w/ ADC)	0.016221	0.999922
3. LSQ (Wheatstone w/ ADC)	0.009198	0.999956
4. SGD Regressor (Wheatstone w/ ADC)	0.030477	0.999854
5. Inverse SGD (Wheatstone w/ ADC)	0.016221	0.999922
6. Random Forest (Wheatstone w/ ADC)	1.161318	0.994451

TABLE V: Low MSE and high R2 is desirable

Table VI presents a small subset of the test data, showing the actual temperature values alongside the predictions from the Random Forest, Inverse LSQ, LSQ, SGD, and Inverse SGD Regressor models for the Wheatstone bridge setup with ADC. The corresponding graph is available in Fig. 4. We thus conclude that the LSQ model is best at predicting the temperature. Thouge the results are provided only for a small range of temperatures in Table VI and Fig. 4 for brevity and visualization respectively, the results hold true for all values of practical interest.

TABLE VI: Wheatstone Validation Data and Predictions

Volt (V)	Actual (°C)	Inv LSQ (ADC)	LSQ (ADC)	SGD (ADC)	Inv SGD (ADC)	RF (ADC)
0.2634	51.9	51.9136	51.9070	51.7938	51.9906	53.3456
0.3127	65.9	65.7972	65.8329	65.7442	65.8829	67.6248
0.2083	37.2	37.1962	37.1906	37.2964	37.1134	37.0563
0.4037	93.1	93.6290	93.4194	93.9231	93.2383	89.0505
0.2092	37.5	37.4305	37.4238	37.5239	37.3513	37.0563
0.2978	61.6	61.5231	61.5485	61.4304	61.6219	61.8901
0.2028	35.8	35.7687	35.7707	35.9127	35.6628	36.7501
0.2666	52.7	52.7931	52.7892	52.6712	52.8748	53.3456
0.3185	67.5	67.4803	67.5183	67.4462	67.5568	68.7954
0.2850	57.9	57.9067	57.9202	57.7921	58.0053	58.6497

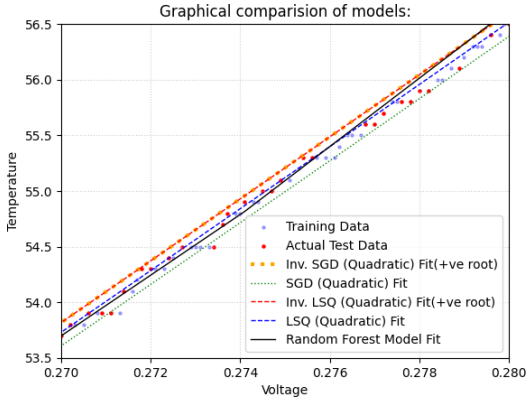

Fig. 4: Comparison of Random Forest, Inverse LSQ, LSQ, SGD and Inverse SGD Regressor predictions against actual test data for the Wheatstone bridge with ADC for temperatures between 53.5°C to 56.5°C

The code for the Arduino, which predicts the temperature from the voltage and flashing instructions are available here https://github.com/gadepall/pt100/codes/arduino/inference

References

[1] T. Bartelt, *Industrial Control Electronics: Devices, Systems & Applications*. Delmar Cengage Learning, 2005.

[2] V. Chudý and F. Janíček, "Identification and prediction of the dynamic properties of resistance temperature sensors," *Sensors and Actuators A: Physical*, vol. 197, pp. 69–75, 2013. [Online]. Available: https://doi.org/10.1098/rsta.1887.0006

[3] C. Liu, C. Zhao, Y. Wang, and H. Wang, "Machine-learning-based calibration of temperature sensors," *Sensors*, vol. 23, no. 17, p. 7347, 2023. [Online]. Available: https://doi.org/10.3390/s23177347

[4] M. Li, C. Zeng, and Y. Wang, "Pt100 temperature measurement sensors design and transducer simulation," *Journal of Physics: Conference Series*, vol. 2897, no. 1, p. 012011, 2024. [Online]. Available: https://doi.org/10.1088/1742-6596/2897/1/012011

[5] A. Samuk and M. Çakır, "New circuit design and implementation for temperature measurement based on pt100 sensor," *ResearchGate*, 2023. [Online]. Available: https://www.researchgate.net/publication/376796536_New_Circuit_Design_and_Implementation_for_Temperature_Measurement_based_on_PT100_Sensor

[6] H. L. Callendar, "On the practical measurement of temperature: Experiments made at the cavendish laboratory, cambridge," *Philosophical Transactions of the Royal Society of London. A*, vol. 178, pp. 161–230, 1887.

[7] H. Robbins and S. Monro, "A Stochastic Approximation Method," *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400 – 407, 1951. [Online]. Available: https://doi.org/10.1214/aoms/1177729586

[8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct 2001. [Online]. Available: https://doi.org/10.1023/A:1010933404324

[9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: http://jmlr.org/papers/v12/pedregosa11a.html

[10] J. Fraden, *Handbook of Modern Sensors: Physics, Designs, and Applications*. Springer, 2016.

[11] T. Mien and N. Hien, "Precision temperature measurement using wheatstone bridge," in *2019 International Conference on System Science and Engineering (ICSSE)*. IEEE, 2019, pp. 145–149. [Online]. Available: https://ieeexplore.ieee.org/document/8823122
[12] A. M. Legendre, "The method of least squares," *Memoires de l'Academie Royale des Sciences*, 1805.
[13] *ADS111x Ultra-Small, Low-Power, I2C-Compatible, 860-SPS, 16-Bit ADCs*, Texas Instruments, 2018, sBAS444D.
[14] Y. M. Jaradat, M. A. Alia, M. Z. Masoud, A. A. Manasrah, I. A. Jannoud, and O. Alheyasat, "Beyond one-size-fits-all: Comparing and selecting regression metrics for robust model assessment," in *2025 12th International Conference on Information Technology (ICIT)*, 2025, pp. 416–422. [Online]. Available: https://doi.org/10.1109/ICIT64950.2025.11049268
[15] Dev47Apps, "Droidcam: Use your phone as a webcam," https://droidcam.app/obs/, 2023, accessed: 2023-10-27.
[16] OBS Project, "Open broadcaster software (OBS) studio," https://obsproject.com/, 2012, software.
[17] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035. [Online]. Available: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf
[18] A. Kaehler and G. Bradski, *Learning OpenCV 3: Computer vision in C++ with the OpenCV library*. " O'Reilly Media, Inc.", 2016.
[19] JaidedAI, "Easyocr: Ready-to-use ocr with 80+ supported languages," https://github.com/JaidedAI/EasyOCR, 2020.

Appendix A

A. Prerequisites

- Install DroidCam [15] on your Android device.
- Install OBS Studio [16] on your Linux laptop.
- Connect your Android phone to your Linux laptop.
- Open DroidCam on your phone. Open OBS studio on your laptop and slect DroidCam device from the menu.
- Install Python Libraries: Install the necessary packages using `pip`:

```
pip install opencv-python easyocr numpy
```

Note: `easyocr` will also install `PyTorch [17]`, which is a large download and a required dependency.

B. Collecting the Dataset

1) Go to the following directory
https://github.com/gadepall/pt100/AutoTrainer/
and run the script

```
python3 a.py
```

2) Two windows will appear: `'Voltage'` and `'Temperature'`. These show the exact (and processed) regions being sent to the OCR engine.
3) To stop the script, make sure one of these windows is active (click on it) and press the 'q' key.
4) This script uses OpenCV [18] and EasyOCR [19] to capture video from a webcam, identify and read text from two specific regions (voltage and temperature in this case), and log this data to a file. It is designed to be calibrated for a specific, fixed-camera setup, such as this case where we are reading data from an LCD screen and a separate standard thermometer.

C. Output

The script generates two types of output:

- `out.txt`: A text file that logs the detected readings. Every 15 frames, if text is found in both regions, a new line is added, formatted as: `[voltage_reading] [temperature_reading]`.
- `img/` Directory: This folder saves `voltageXX.png` and `tempXX.png` images. These are the exact frames that were sent to EasyOCR. If you are getting bad readings, check these images to see why. They are perfect for debugging your calibration and fixing the readings.
The directory structure will look similar to this:

```
codes/
|-- out.txt
|-- img/
|   |--voltage_1.png
|   |--temperature_1.png
|   |--voltage_2.png
|   |--temperature_2.png
|   |--voltage_XX.png
|   |--temperature_XX.png
```

`out.txt` may not be perfect due to some instances of incorrect character recognition. In such case, the data may have to be manually updated and saved in `trainingdata.txt`.
- The final training data is available in the following text file
https://github.com/gadepall/pt100/blob/main/AutoTrainer/trainingdata.txt
This was obtained from out.txt after manually modifying some of the incorrect entries.

D. Important: Calibration is Required!

This script will not work correctly without calibration. Following are the adjustments that are necessary in `a.py`:

1) GPU Usage: If you do not have a compatible NVIDIA GPU, you must change this line

```
# From:
reader = easyocr.Reader(['en'], gpu=True)

# To:
reader = easyocr.Reader(['en'], gpu=False)
```

The script will run much slower on the CPU but will still function.

2) Frame Cropping: This is the most critical part. The script has to be told where exactly to look for the data. In the following, the parameters $h/2, w/3$ may need to be modified.

```
# Change the crop values according to requirement
voltage_frame = frame[0:int(h/2), :]
temp_frame = frame[int(h/2):, 0:int(w/3)]
```

In the above snippet, `voltage_frame` is currently set to the top half of the entire camera feed. `temp_frame` is set to the bottom-left third of the feed. These pixel/percentage

values to need to be changed to precisely isolate the voltage and temperature readings. The `Voltage` and `Temperature` windows show what the script sees in these cropped zones, so the values may be adjusted accordingly and re-run until they are correct.

3) Image Processing: The following code snippet for reading the physical thermometer may have to be modified to fit the specific camera, lighting, and display setup.

```
# These values need to be calibrated according to
    requirement
gray_t = cv2.cvtColor(temp_frame, cv2.COLOR_BGR2GRAY)
t_lower = 50
t_upper = 100
_, thres = cv2.threshold(gray_t, 130, 255, cv2.
    THRESH_BINARY_INV)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (11, 11))
closed_img = cv2.morphologyEx(thres, cv2.MORPH_CLOSE, kernel
    )
pretemp = cv2.bitwise_not(closed_img)
temp = cv2.GaussianBlur(pretemp, (17, 17), 0)
```

Table VII lists some of the values that need to be adjusted.

`cv2.threshold(gray_t,130, 255, cv2.THRESH_BINARY_INV)`	130 is the threshold value. This is highly sensitive to lighting. Adjust it up or down until the text is clearly separated from the background in the `'Temperature'` window.
`cv2.getStructuringEleme nt( cv2.MORPH_RECT, (11,11))`	This is the kernel size for the `'closing'` operation, which helps connect broken parts of a number. You may need to make it larger or smaller.
`cv2.GaussianBlur(pretemp, (17,17), 0)`	This is the kernel size for the blur.

TABLE VII

We need to look at the `Temperature` window while calibrating. The goal is to make the numbers we want to read clear and solid, while removing as much background noise as possible. It is also preferred to hide the decimal point, either physically or by further image processing.